HACK4HACK CREW HIRING CHALLENGE 2026 · PROBLEM STATEMENT 1



LAST SEAT

Race-proof team formation.

Fifty people click Join on one last seat. Exactly one gets it, every time, on any number of servers, because the database is the only referee.

122 API tests on real Postgres

9 browser tests

p95 114.1 ms at 100 users

8 replica kills, 0 violations

Gaurav Tiwari · github.com/Gauravtiwari31/Race-proof

THE PROBLEM

Fifty clicks. One last seat

Most team-formation code reads the member count, checks it, then writes. Between the read and the write, everyone else reads the same count. Under real concurrency, a full team keeps accepting people.

We ran both versions against the same database in the app's Race Arena: 50 real requests, released together.

NAIVE CODE



people got one seat

member_count no longer matches the members

LASTSEAT



winner, every run

49 clean refusals: 409 TEAM_FULL

Every guarantee: enforced, proven, visible



Enforced by Postgres

One transaction per change. Conditional UPDATE, primary keys, a partial unique index and CHECK constraints are the referee, not application code.

`service.ts · 0001_schema.sql`



Proven by a test

122 API tests run on real Postgres with no mocked data layer. Race tests release 50 connections at a barrier, and a mutation check proves they can fail.

`race.last-seat-50 · race.mutation-check`



Visible in the product

The Race Arena runs 50 bots against the real database in one click. A naive mode shows the bug the tests guard against.

`/arena · /ops`

One statement, no gap between check and act

```
UPDATE teams
   SET member_count = member_count + 1,
       version = version + 1
 WHERE id = $1
   AND member_count < max_size
RETURNING id, member_count, version;
```

1 **Racers queue on the team's row lock.**

The first UPDATE locks the row; the other 49 wait.

2 **The winner commits.**

READ COMMITTED re-evaluates each waiter's WHERE against the new row.

3 **Everyone else updates 0 rows.**

No row returned means 409 TEAM_FULL. Nothing to roll back by hand.

`PRIMARY KEY (hackathon_id, user_id)`

ONE TEAM PER PERSON

`UNIQUE INDEX … WHERE role = 'leader'`

EXACTLY ONE LEADER

`CHECK (member_count BETWEEN 0 AND max_size)`

LAST LINE OF DEFENCE

Each guarantee, what holds it up, what proves it

Guarantee	Enforced by	Proven by
Exactly one person gets the last seat	Conditional UPDATE in one statement	`race.last-seat-50, race.mutation-check`
One team per person, per hackathon	memberships primary key + composite FK	`race.user-two-teams, rules.composite-fk`
Always exactly one leader	Partial unique index + team lock	`race.leader-and-member-leave`
No deadlocks	One lock order: team, members, user, requests	`race.mixed-no-deadlock, race.lock-order-*`
Authorisation cannot go stale	Leader check in a new statement after the lock	`race.stale-leader-check`
No request left pending by a join	User row as a per-person mutex	`race.request-vs-join`
Retries apply once	Idempotency key row in the same transaction	`idem.*, chaos.fault-rollback`
A crash loses nothing	One transaction per mutation	`chaos.sh: replicas killed mid-storm`

PROVEN, NOT PROMISED

Every test was seen to fail on the bug it guards

A test that cannot fail proves nothing. Each fix below was undone on purpose, and the named test went red against real Postgres.

Bug put back	Test	What it reported
Read-then-write seat	`race.mutation-check`	Several winners for one seat
Accept closes the request before adding the member	`race.lock-order-accept-vs-redeem`	A real deadlock (500) where 409 was due
Redeem locks the code before the team	`race.lock-order-redeem-vs-delete`	A real deadlock (500) where 404 was due
Leader check inside the locking statement	`race.stale-leader-check`	A demoted leader removed someone (200)
No per-person lock on join vs request	`race.request-vs-join`	A request stayed pending after its author joined

122

API tests, real Postgres

9

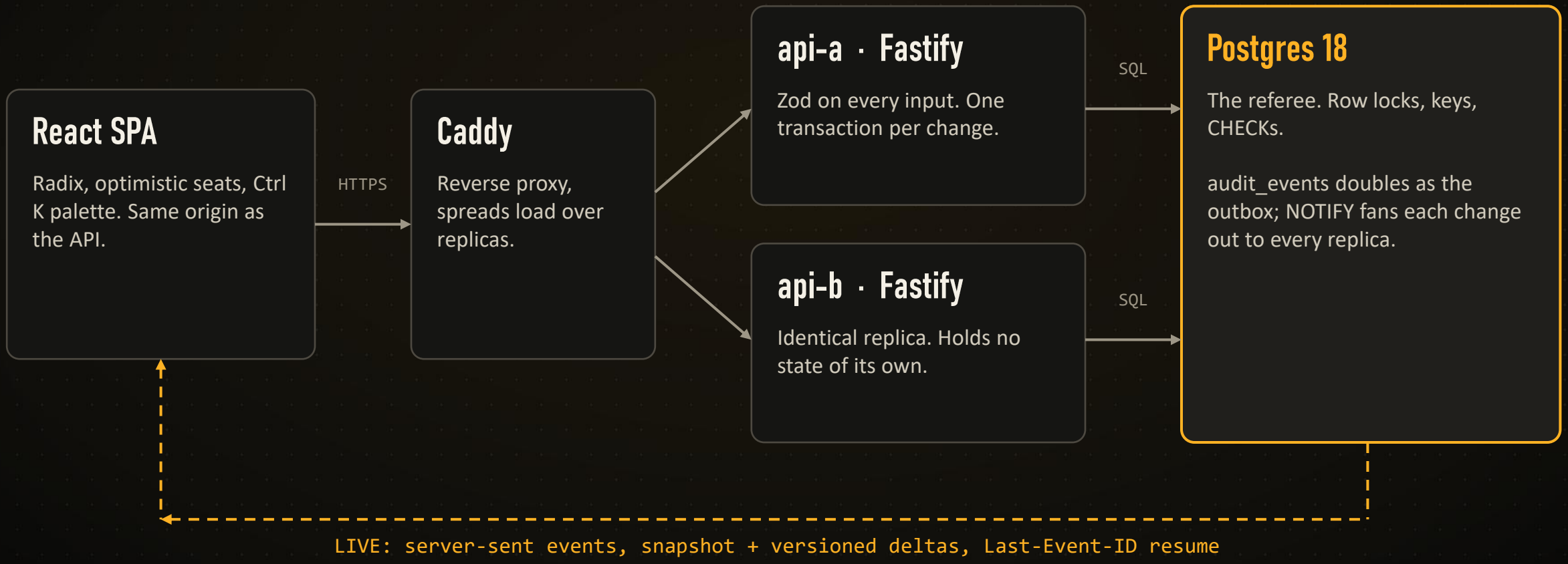
browser tests, axe + keyboard

20×

last-seat race, 2 processes

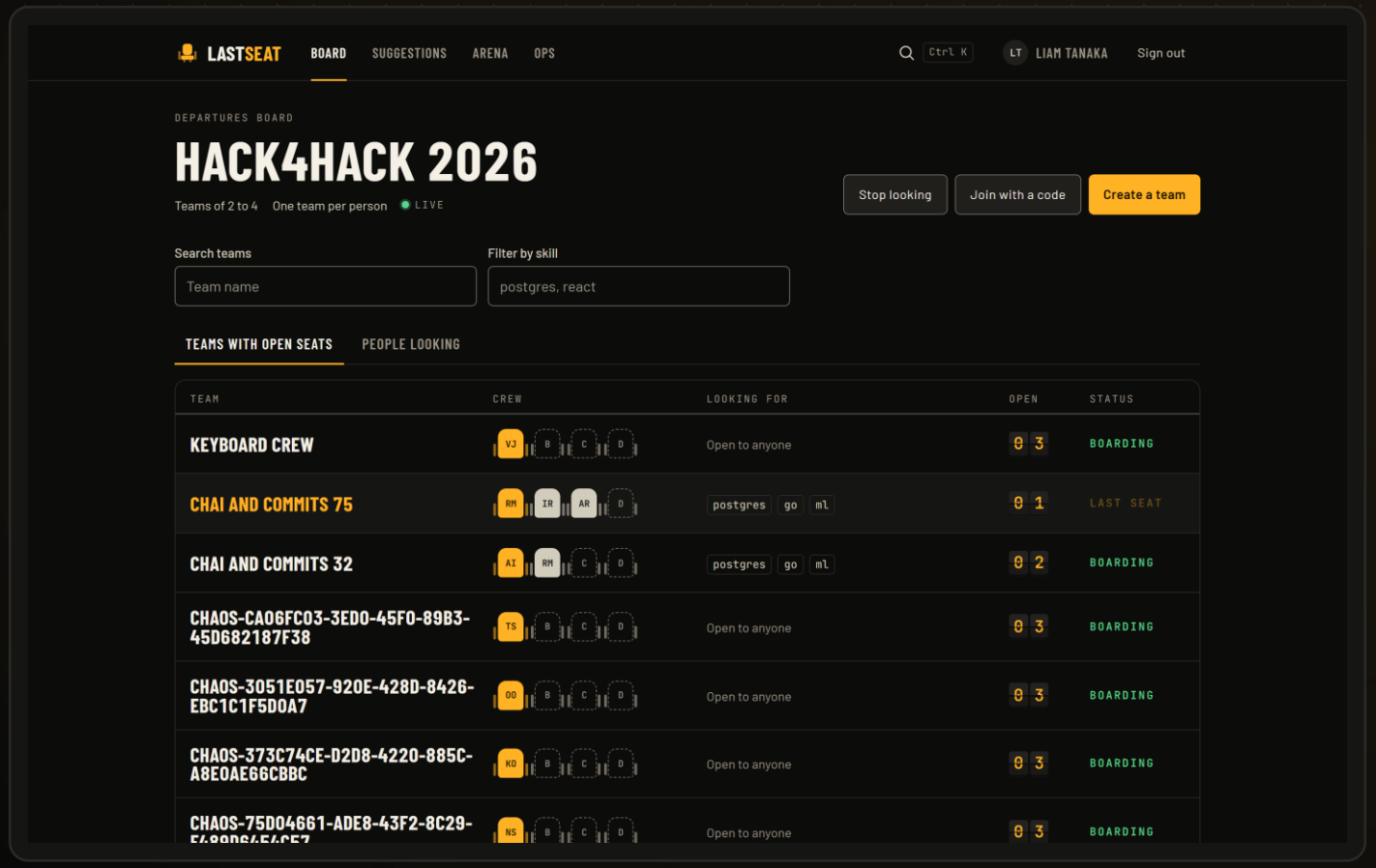
ARCHITECTURE

One database referee, any number of servers



Why not Redis, SERIALIZABLE or WebSockets? One ACID database keeps every side effect in one commit; updates only flow one way.

A departures board for hackathon teams



Live board and team pages

Seats change everywhere without a refresh.

Invites and join requests

Expiring, revocable codes; leaders accept or decline.

Optimistic, and honest

A seat appears instantly, rolls back with the reason.

Explained suggestions

Missing skills and time-zone overlap, scored openly.

Audit time machine

Replay any team’s history from the audit log.

Phone, keyboard, axe

360 px, Tab-only flows, Ctrl K everywhere.

Judges can run the race themselves

1. **Open /arena and press Unleash 50 bots.**

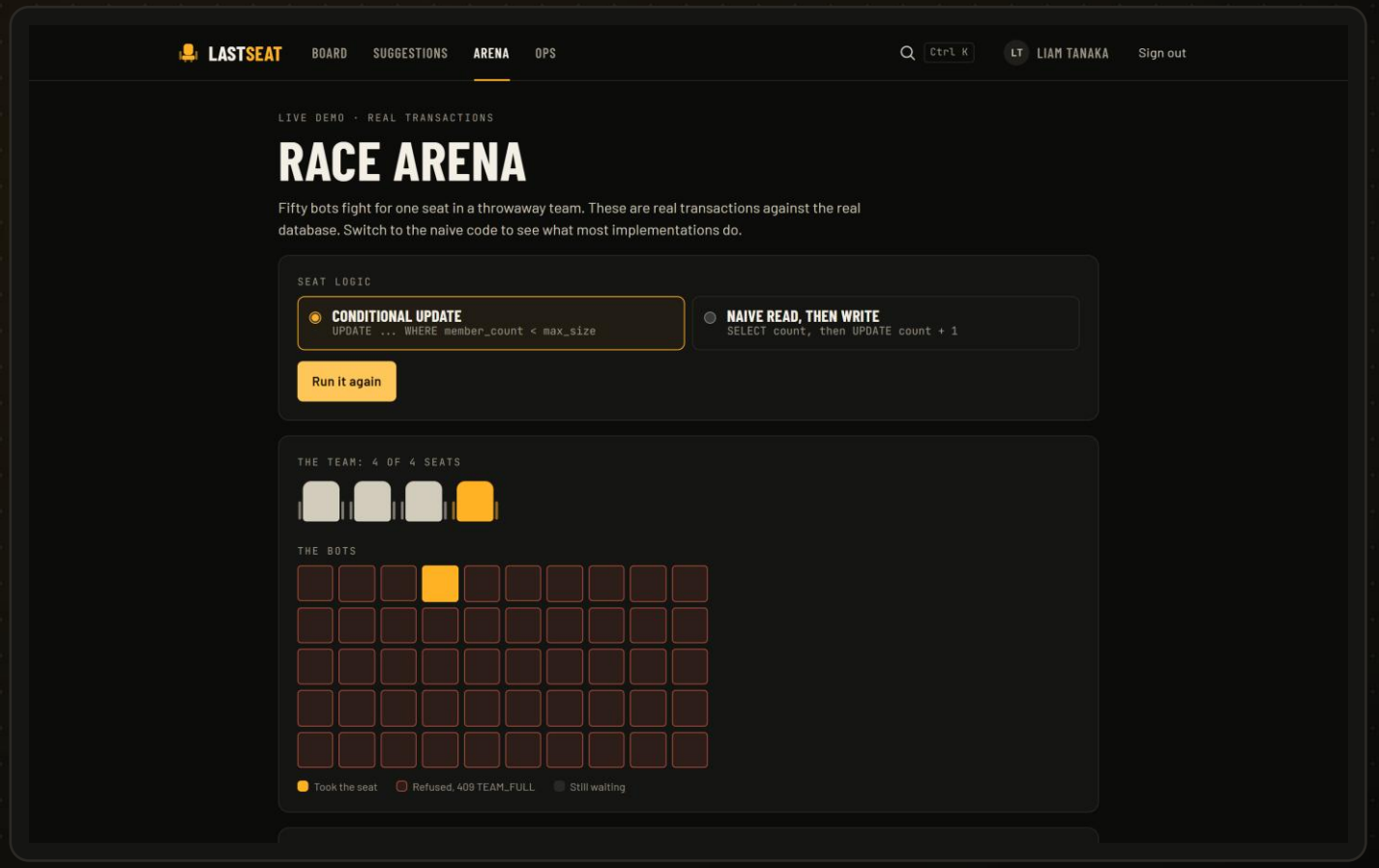
   50 real transactions hit a throwaway team with one seat left.

2. **One amber winner, 49 refusals.**

   Each refusal is a real 409 TEAM_FULL from the database.

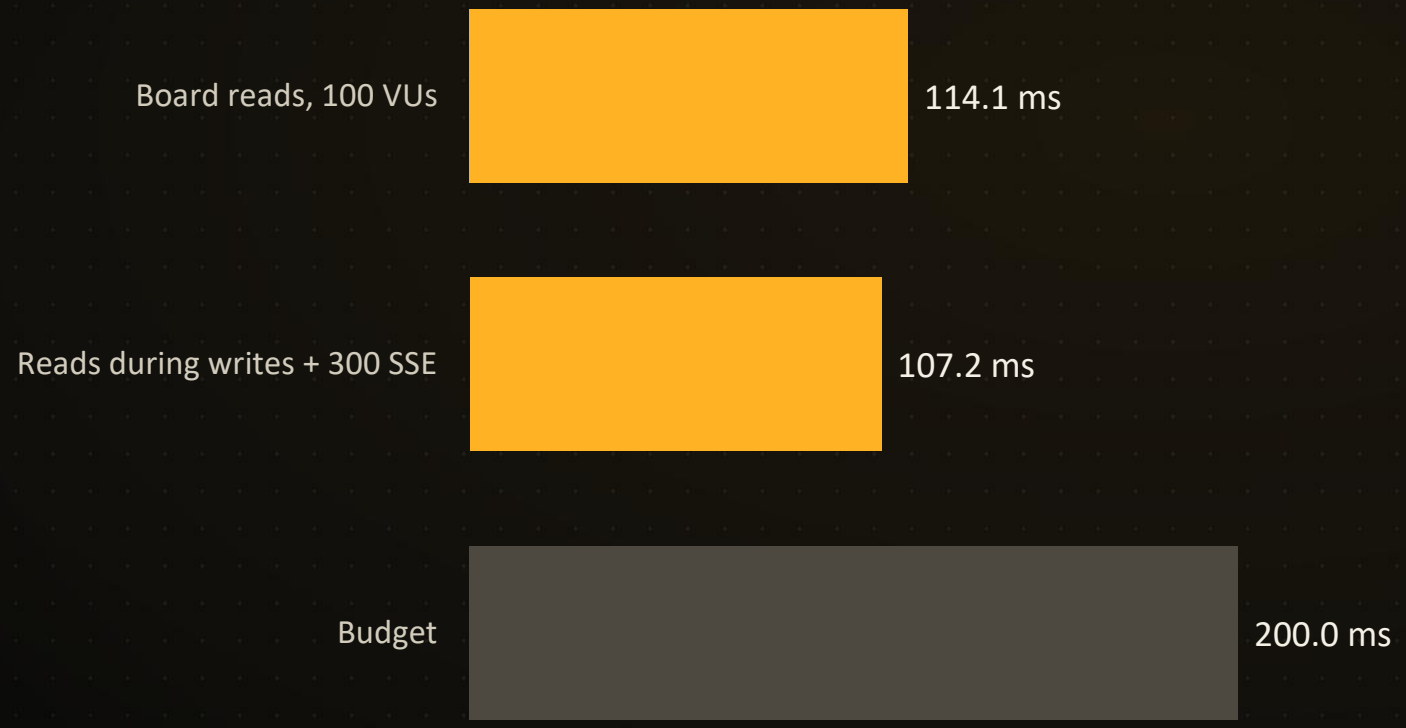
3. **Switch to the naive mode, run again.**

   4 winners in our run, and a corrupted member count.



Sandboxed: its own hackathon, its own small connection pool, rate limited, excluded from the invariant checks.

Board p95 114.1 ms against a 200 ms budget



326,379

board requests, 0 failed

1,811/s

sustained throughput

10,000

users and 2,000 teams seeded in ~1.5 s

Local Compose: two replicas behind Caddy, k6 on the same laptop. Board reads use a micro-cache keyed by board version; the mixed run keeps invalidating it, so it is the fairer number.

Kill a server mid-join. Nothing is lost

8

replicas SIGKILLed

1,280

seats taken in the storm

0

invariant violations

Thirty virtual users create, join and leave teams for a minute while a replica is killed every five seconds. Afterwards the same invariant checker the tests use runs over the whole database.

It holds because every mutation is one transaction: a dropped connection rolls back, and the client retries with the same key.



Idempotent retries

Each write may carry an Idempotency-Key. The key row commits in the same transaction as the change, so a crash rolls back both, and a duplicate waits on the key, then replays the stored answer.

`idem.*` · `chaos.fault-rollback`

SECURITY

Private by default, checked after the lock



Leader-only actions

Accept, decline and remove re-check the role in a new statement after the team lock.



Unguessable invite codes

50 random bits, shown once, stored only as SHA-256; expiry, revocation, max uses.



Durable rate limits

Codes, logins, sign-ups and writes; in Postgres, shared by replicas, burst-proof.



Sessions and passwords

Argon2id; 256-bit tokens stored hashed; httpOnly, SameSite cookies; CSRF header.



Validated input

Zod on every body, query and parameter. Errors never carry a stack trace.



Append-only audit log

Written inside each mutation; the app's database role cannot update or delete it.

DECISIONS

Trade-offs we chose, and the next week

TRADE-OFFS

Denormalised member count

CHECKED

Fast board reads; one transaction and an invariant checker keep it true.

Server-sent events, not WebSockets

ON TIME

Updates flow one way; reconnect and resume come for free.

Rate limits in Postgres, not Redis

ON TIME

Durable and shared; costs one write per attempt.

Fixed-window limiter

NOTED

Simple and durable; up to 2× the limit across a boundary.

WITH ONE MORE WEEK

Commit-ordered event log

SCHEDULED

Logical replication instead of id order for the live feed.

Waitlist with seat holds

SCHEDULED

Offer a freed seat to the next person for ten minutes.

Organiser console

SCHEDULED

Create hackathons, set team sizes, export teams.

WebAuthn and audit partitioning

SCHEDULED

Passkeys; audit table partitioned per hackathon.

TRY IT

FIFTY CLICKS ONE SEAT ONE WINNER



Live demo	https://lastseat-b4mi.onrender.com
Code, tests, design doc	github.com/Gauravtiwari31/Race-proof
Demo sign-in	user1@seed.local / demo-password-123
Run it locally	docker compose up -d --build → localhost:8080